Aegora Isle — Governance Regimes & KPI Reference Guide

Plain-language, code-grounded explainer of what G1/G2/G3 actually do

Aegora Isle Project

2026-07-01

Contents

Governance Regimes & KPI Reference Guide **1**
1. The world, in one paragraph . . . 1
2. The Three Governance Regimes . . . 2
   2.1 G1 — Ungoverned (the control group) . . . 2
   2.2 G2 — Rule-Based (the static government) . . . 2
   2.3 G3 — Adaptive RL (the AI government) . . . 3
3. The Four Scenarios . . . 7
4. What happens in one cycle . . . 8
5. The Three Headline Metrics, in Detail . . . 9
   5.1 Gini coefficient — what inequality looks like as one number . . . 9
   5.2 Depletion rate — how fast the lagoon is shrinking, per cycle . . . 9
   5.3 Resource stock — the lagoon's current raw-resource quantity . . . 9
6. Quick reference — file map . . . 10

Governance Regimes & KPI Reference Guide

A plain-language, code-grounded explanation of what G1/G2/G3 actually do, what the four scenarios mean, what happens inside one simulation cycle, and exactly what the three headline metrics (Gini, depletion rate, resource stock) represent. Every number and formula below is copied directly from the implementation — file paths are given throughout so you can verify anything here against the source.

This is a companion to `G1_PLAN.md` / `G2_PLAN.md` / `G3_PLAN.md` (which are *build* plans for an engineer implementing the system) and `DECISIONS.md` (which records *why* each ambiguous spec point was resolved the way it was). This document is the *what does it actually do and why does it matter* explainer.

---

1. The world, in one paragraph

Twenty (or more) agents live on an island with one shared lagoon. The lagoon holds a **raw resource**. **Extractors** pull raw resource out of the lagoon into their own inventory. **Producers** are supposed to turn raw resource into **food** and **tool** goods (a known, currently-broken pipeline — see the callout in §3). **Traders** buy and sell food/tool on a market whose price rises as the lagoon empties. **Governors** exist to intervene — but *how* they intervene is entirely determined by which of three governance regimes is active. The lagoon regenerates a little every cycle, and is periodically hit by random disruption events that cut its stock by 10–30%. If it runs dry, the simulation collapses.

The central research question this whole platform exists to answer: **which kind of governance — none, static rules, or a learned AI policy — keeps the island prosperous and equal under different conditions?**

2. The Three Governance Regimes

All three regimes share one interface (`aegora_isle/governance/governance_module.py`):

```
on_action(agent_id, action) -> (modified_action, reward_delta)
on_step(env_state) -> None
```

`on_action` fires for **every single agent's turn**, every cycle — it can modify the action before it's applied and add a bonus/penalty to that agent's reward. `on_step` fires **once per cycle**, after all agents have acted — it's where world-level interventions (redistribution, training) happen. The environment itself never knows which regime is active; it just calls these two methods. This is what makes the three regimes a fair scientific comparison — same environment code, same agents, only the governance module differs.

2.1 G1 — Ungoverned (the control group)

File: `aegora_isle/governance/configs/g1_ungoverned.yaml`

```yaml
regime: G1
```

That's the entire config — G1 has zero parameters. Internally it's `G1Handler` (`governance_module.py`), and both its methods are no-ops:

```python
def on_action(self, agent_id, action):
    return action, 0.0      # action passes through completely unchanged

def on_step(self, env_state):
    return None              # nothing happens at the end of a cycle
```

Population: no Governor agents exist at all. At 20 agents the split is roughly 7 Extractors / 7 Producers / 6 Traders (the Governor's would-be 25% share gets redistributed across the other three types — see `AegoraEnv._agent_counts()`).

What it models: laissez-faire / "tragedy of the commons." Every Extractor pulls as much raw resource as it wants, every cycle, with no limit and no consequence. This is the baseline every other regime is measured against — if G2 or G3 can't outperform G1, governance isn't adding value.

2.2 G2 — Rule-Based (the static government)

File: `aegora_isle/governance/configs/g2_rule_based.yaml`

```yaml
regime: G2
extraction_cap: 3.0           # max raw units one Extractor may pull per cycle
sanction_probability: 0.85    # P(penalty fires | an Extractor exceeds the cap)
sanction_magnitude: -2.0      # reward penalty applied when a sanction fires
redistribution_period: 100    # redistribute every N environment timesteps
redistribution_rate: 0.10     # fraction of Producer+Trader cash redistributed
```

Implemented in `aegora_isle/governance/rule_based.py` (`RuleBasedGovernor`). This is a fixed, never-changing rule-set — nothing here adapts as the simulation runs.

Mechanism 1 — extraction cap (runs inside `on_action`, every Extractor turn):

Each Extractor's action includes an `amount` field, a fraction in `[0,1]`. The environment converts that fraction into raw units by multiplying by `MAX_EXTRACT_AMOUNT = 5.0` (a fixed environment constant — the absolute maximum *anyone* could ever request). G2 checks the requested amount against its own, *tighter* `extraction_cap = 3.0`:

```
requested = amount_fraction * 5.0

if requested <= 3.0:
```

```
    pass through unchanged, no penalty
else:
    clip to cap_frac = 3.0 / 5.0 = 0.6   (i.e. only 3.0 raw units go through)
    roll a die: with probability 0.85, apply a -2.0 reward penalty
```

Worked example: an Extractor requests `amount = 1.0` (the maximum possible, 5.0 raw units). G2 clips this down to `0.6` (3.0 raw units — all that's actually extracted), and 85% of the time also subtracts 2.0 from that Extractor's reward for that turn, as a deterrent.

Mechanism 2 — scheduled redistribution (runs inside `on_step`, once every 100 timesteps):

```
total_pool = 0
for every Producer and Trader:
    contribution = their_cash * 0.10
    subtract contribution from their cash
    total_pool += contribution

per_extractor_share = total_pool / number_of_extractors
give per_extractor_share to every Extractor
```

This is a wealth tax on Producers/Traders, redistributed equally among Extractors — a crude universal-basic-income mechanism. It conserves total cash exactly (nothing is created or destroyed, just moved).

What it models: a non-adaptive bureaucracy. A human policy-maker could have written these exact rules. The doctoral hypothesis (H3) specifically predicts that an *adaptive* AI governor (G3) should outperform this static rule-set in volatile conditions, where a fixed cap can't react to a sudden disruption event the way a learned policy could.

2.3 G3 — Adaptive RL (the AI government)

This is the most involved regime — a real reinforcement-learning agent that *learns* what caps, redistribution rates, and sanction probabilities to set, rather than having them hand-tuned in a YAML file.

2.3.1 What model, what library

Library	Stable-Baselines3 (`stable-baselines3>=2.0.0`, pinned in `pyproject.toml`; this project uses 2.9.0)
Underlying deep-learning framework	PyTorch (`torch`) — SB3 is built on top of it
Algorithm	**SAC** — Soft Actor-Critic
Policy network	`MlpPolicy` — a plain feed-forward neural network (no images, no convolutions — the input is a 15-number vector, so a simple multi-layer perceptron is the right tool, not anything fancier)

Why SAC and not PPO (PPO is what trains the Extractor/Producer/Trader agents — see §2.3.6):

1. **Continuous output.** The Governor's decision (cap level, redistribution rate, sanction likelihood) is a continuous number, not a discrete choice. SAC is built natively for continuous action spaces.
2. **Sample efficiency.** SAC is *off-policy* — it learns from a memory bank (the **replay buffer**) of past experience, re-using old transitions many times. The Governor only gets to make one decision per *cycle* (not per individual agent turn), so experience is scarce; squeezing more learning out of each data point matters here.
3. **Built-in exploration.** SAC has an explicit "entropy" term in its objective that rewards the policy for *not* collapsing onto one fixed action too early. Without this, a learned Governor could plausibly get stuck always picking the same boring cap value forever.

2.3.2 The action space — what SAC actually outputs A **3-dimensional continuous box**, each value constrained to `[0, 1]`:

`[cap_norm, redist_norm, sanction_probability]`

These three raw numbers get mapped into real-world ranges (from `g3_adaptive_rl.yaml`):

Output	Range it maps into	Meaning
`cap_norm`	`[0.5, 5.0]` (`extraction_cap_range`)	The extraction cap, same concept as G2's fixed `3.0`, except SAC chooses this number itself, every cycle
`redist_norm`	`[0.0, 0.3]` (`redistribution_rate_range`)	The redistribution rate, same concept as G2's fixed `0.10`
`sanction_probability`	used directly, `[0, 1]`	Probability that an over-cap Extractor gets sanctioned this cycle

The mapping formula (`AdaptiveRLGovernor._unbox`):

```
real_value = lo + clip(sac_output, 0, 1) * (hi - lo)
```

Important design decision (documented in `DECISIONS.md` "G3 SAC action space"): the Governor *agent's* actual action space defined in `aegora_isle/agents/governor.py` is a much bigger, more complex `Dict` (`action_type`, `value`, `target_agent_idx` — the last one alone spans 500 possible target agents). SAC does **not** learn on that full space. A design review found that flattening the full Dict into one SAC action would bury the 3 numbers that actually matter inside ~500 nearly-always- irrelevant dimensions (for picking *which* agent to sanction), making learning much harder for no real benefit. So G3 deliberately uses this simpler, purpose-built 3-number action space instead — "who gets sanctioned" is decided by a simple rule (the Extractor that just violated the cap, inside `on_action`), not learned by SAC at all.

2.3.3 The observation space — what SAC sees A **15-dimensional vector**, reused directly from `aegora_isle/agents/governor.py`'s `Governor.get_observation()` (the same construction a Governor *agent* would see — G3 doesn't duplicate this logic, it borrows a live Governor agent's own observation method):

```
[gini, hdi_proxy, gdp_indexed, depletion_rate, resource_stock_normalised,
 *wealth_histogram (10 bins)]
```

Index	Field	Range
0	Gini coefficient (instantaneous)	`[0, 1]`
1	HDI proxy	`[0, 1]`
2	GDP indexed (raw per-cycle production+trade value)	`[0, 1e6]`
3	Depletion rate (fractional, live form)	`[-1, 1]`
4	Resource stock, normalised to `stock/initial_stock`	`[0, 1]`
5–14	Wealth histogram — fraction of the population in each of 10 wealth bins	`[0, 1]` each

2.3.4 The reward — how SAC is praised or punished This is the composite reward from spec §2.2.2, computed once per cycle in `AdaptiveRLGovernor.compute_composite_reward()`:

```
reward = 0.40 * equality + 0.35 * sustainability + 0.25 * productivity
```

Three components, each computed fresh every cycle:

Equality — `1 - gini`. Higher when wealth is more evenly distributed. If everyone has equal wealth, `gini = 0`, so `equality = 1.0` (max reward contribution). If one agent has everything, `gini` approaches `1`, so `equality` approaches `0`.

Sustainability — `stock_norm * max(0, 1 - depletion_rate)`. Two things multiplied together: how full the lagoon currently is (`stock_norm`, 0 = empty, 1 = full), and how slowly it's being depleted (`1 - depletion_rate`, where `depletion_rate` is a small fraction; positive means the pool is shrinking). If the pool is depleting fast, this term gets clamped to `0` rather than going negative.

Productivity — `log(1 + gdp_indexed)`. The log compresses the scale so a huge GDP swing doesn't completely dominate the other two terms; `+1` inside the log avoids taking `log(0)`.

Normalisation (the `_RunningNormaliser` class, Welford's online algorithm): each of the three raw components above gets its own running mean and standard deviation, updated every cycle from every value it's ever seen. Before being multiplied by its weight, each component is converted to a "how many standard deviations from its own historical average is this, right now" score, then clipped to `±5`. This matters because the three raw components live on wildly different natural scales (equality is always in `[0,1]`; productivity's `log(1+gdp)` could range much wider) — without normalising, the biggest-scale term would silently dominate the weighted sum regardless of its weight.

Where the reward goes: it's pushed into SAC's **replay buffer** along with the observation and the action that produced it — *not* read directly by anything else. Training (below) samples batches from this buffer to update the network.

Punishment for individual agents (separate from the composite reward above — this is the per-Extractor penalty, mirroring G2): inside `on_action`, if an Extractor's request exceeds the *current* SAC-set cap (`self.current_cap`, which changes every cycle), the action is clipped exactly like G2's mechanism, and with probability `self.current_sanction_probability` (also SAC-set, not fixed), that Extractor's reward for that turn gets `sanction_default_magnitude = -2.0` added to it.

2.3.5 Hyperparameters — every one, explained From `g3_adaptive_rl.yaml`'s `sac:` block, passed directly into SB3's `SAC(...)` constructor:

Hyperparameter	Value	What it controls
`learning_rate`	`3.0e-4` (0.0003)	How big a step the network's weights move on each training update. Standard default for SAC.
`buffer_size`	`100000`	Maximum number of past (observation, action, reward) transitions kept in memory. Older ones get overwritten once full.
`batch_size`	`256`	How many stored transitions get sampled at once for each training update.
`gamma`	`0.99`	Discount factor — how much future reward matters vs. immediate reward. 0.99 means the agent cares about consequences quite far into the future (close to 1 = far-sighted).

Hyperparameter	Value	What it controls
`tau`	`0.005`	How fast SAC's "target network" (a slowly-updated copy used to stabilise training) tracks the main network. Small value = very slow, smooth tracking.
`ent_coef`	`"auto"`	The entropy/exploration weight. `"auto"` means SB3 tunes this automatically during training rather than using one fixed number — recommended default for SAC.
`train_freq`	`[4, "step"]`	How often a training update happens, expressed as "every 4 steps." (Loaded from YAML as a list; SB3's constructor actually requires a tuple — `AdaptiveRLGovernor.__init__` converts this automatically, see `DECISIONS.md` "Manual rollout collector corrections.")
`gradient_steps`	`1`	How many gradient-descent updates happen each time training triggers.
`learning_starts`	`1000`	SAC won't train at all until this many transitions have been collected — before that, it just acts randomly to build up a diverse memory bank ("warm-up").
`target_update_interval`	`1`	How often the target network gets updated, in training-update units.
`policy_kwargs.net_arch`	`[256, 256]`	The neural network's hidden-layer shape: two fully-connected layers of 256 neurons each, for both the actor (decision-maker) and critic (value-estimator) networks.

2.3.6 How training actually happens, step by step Unlike a typical SB3 tutorial (`model.learn(total_timesteps=N)`), this project drives SAC **manually**, because the Governor's decision frequency (once per environment cycle) doesn't match SB3's usual one-decision-per-env-step assumption, and because three *other* models (PPO for Extractor/Producer/ Trader, see `train_agents.py`) need to share the same environment at the same time. Inside `AdaptiveRLGovernor.on_step()`, every single cycle:

1. **Read the current state.** Borrow a live Governor agent's `get_observation()` to build the 15-number observation vector.
2. **Compute the reward** for *this* cycle, using the formula in §2.3.4.
3. **If this isn't the very first cycle**, push a complete transition — *(previous observation, previous action, this cycle's reward, this observation, is-this-terminal)* — into the replay buffer. (This is a one-cycle-delayed pairing: you don't know how good an action was until you see the *next* state and reward.)
4. **Pick a new action** for this cycle: if fewer than `learning_starts` (1000) transitions have been collected so far, pick *randomly* (matching SB3's own warm-up convention);

otherwise, ask the actor network for an action.

5. **Apply the decision**: convert the 3 raw SAC numbers into `current_cap`, `current_redist_rate`, `current_sanction_probability` — these become the values every Extractor's `on_action` check and the redistribution mechanism use for the *next* cycle, until SAC decides again.
6. **Redistribute**, using exactly G2's mechanism, but reading `current_redist_rate` (which changes every cycle) instead of a fixed YAML number.
7. **Train**, but only if the replay buffer already has at least 1000 transitions *and* the cycle count is a multiple of `train_freq`'s `4`. Training means: sample a batch of 256 past transitions, compute how wrong the actor/critic networks currently are, and nudge their weights to be less wrong.

This repeats every single cycle, for as long as the simulation runs. The *specific* cap/redistribution/sanction numbers used right now are always whatever SAC decided last cycle — they are never fixed, unlike G2.

---

3. The Four Scenarios

Defined in `aegora_isle/experiments/scenarios.yaml`. Each scenario sets the same six parameters to different values, forming a 2×2 grid: **resource abundance** (high/low) crossed with **event frequency** (low/high).

Scenario	`initial_stock`	`regeneration_rate`	`event_rate`	`critical_min`	Reading
A	10,000	0.05	0.5	100	High abundance, calm — the "easy" cell
B	10,000	0.05	3.0	100	High abundance, volatile — shocks hit a deep reserve
C	3,000	0.01	0.5	50	Scarce, calm — slow-burn pressure
D	3,000	0.01	3.0	50	Scarce, volatile — the "hard" cell, used for the M1 collapse smoke test

All four share `scarcity_alpha: 0.5` and `base_prices: {food: 10.0, tool: 20.0}` — only the resource dynamics differ between scenarios, not market pricing.

What each parameter does (`aegora_isle/environment/resource_pool.py`):

- **`initial_stock`** — the lagoon's starting (and maximum possible) quantity of raw resource.
- **`regeneration_rate`** — each cycle, the pool grows by `regeneration_rate * stock * (1 - stock/initial_stock)` (logistic growth — fastest around the midpoint, tapering off as it approaches `initial_stock`, never overshooting it).
- **`event_rate`** — expected number of random disruption events per 100 cycles. Each disruption, when it fires, cuts the *current* stock by a random 10–30%.
- **`critical_min`** — if `stock` ever falls below this, the environment considers the run **collapsed** and the episode ends (`ResourcePool.is_collapsed()`).

Scenario A is the scientific baseline — the doctoral hypothesis H1 ("G1 Gini > G2 Gini > G3 Gini") is specifically tested there, because it's the cleanest cell (no scarcity pressure confounding the governance comparison).

Scenario D is the stress test — both M1's AC03 smoke test and the governance-comparison charts in this session deliberately use it because it's the cell most likely to expose a broken mechanism (if a bug exists, D will surface it fastest).

---

4. What happens in one cycle

A "cycle" = every agent in the population gets exactly one turn, in a fixed order (all Extractors, then all Producers, then all Traders, then all Governors). This is enforced by PettingZoo's AEC protocol — see `MERMAID.md` for the full sequence diagram. Concretely, per `aegora_isle/environment/aegora_env.py`'s `step()`:

During the cycle (once per agent's individual turn):

1. The current agent's chosen action is handed to `governance_module.on_action(agent_id, action)` first. This is where G2/G3's cap-clipping and sanctioning happen — the action that actually executes may be different from what the agent originally requested.
2. The (possibly modified) action executes: an Extractor pulls raw resource from the pool into its own inventory; a Producer attempts to convert raw into food/tool (see the callout below); a Trader buys or sells on the market; a Governor's action is queued, not applied immediately.
3. That agent's reward for this turn is computed and recorded.

Once every agent has had a turn (end of cycle):

4. The environment's internal timestep counter increments by 1.
5. This cycle's total production value + trade value is finalised as `gdp_raw` for the cycle (the basis for the GDP-equivalent KPI).
6. `resource_pool.step()` runs: logistic regeneration is applied, then a disruption event is rolled (and, if it fires, immediately cuts the stock).
7. The stock history buffer is updated, and every live KPI (Gini, HDI proxy, GDP, depletion rate, wealth histogram) is **recomputed from scratch** using the state as of right now — this is what feeds every Governor agent's observation *and* G2/G3's decision-making for the next cycle.
8. `governance_module.on_step(env_state)` runs — this is where G2's scheduled redistribution check happens, and where **all** of G3's SAC machinery (§2.3.6) fires: replay buffer update, new decision sampled, redistribution applied, training triggered if due.
9. The environment checks: has the pool collapsed (`stock < critical_min`), or has the cycle limit (`max_timesteps`) been reached? If either is true, every agent is marked truncated and the episode is over.

So, concretely, what changes from one cycle to the next?

- `resource_pool.stock` — down from extraction, down (sharply) from any disruption event that fired, up (slightly) from regeneration.
- Every agent's `inventory/cash` — shifted by whatever they extracted/produced/traded/were sanctioned or redistributed this cycle.
- The five live KPIs — recalculated fresh, reflecting the new state.
- Under G2: nothing changes *unless* the cycle count is a multiple of `redistribution_period` (100) — then cash moves once.
- Under G3: `current_cap`, `current_redist_rate`, and `current_sanction_probability` are **re-decided every single cycle** — this is the core behavioural difference from G2, where those three numbers never move.

Resolved (was previously a known gap; see `DECISIONS.md` "Producer raw-material acquisition — resolved"): step 2's Producer conversion now actually succeeds. When a Producer attempts to produce and lacks enough raw material, it automatically buys the shortfall from a randomly-picked Extractor at the current scarcity price, paid in cash — the same peer-to-peer trade mechanism Traders already use for food/tool, just applied to raw material too. No new agent action was added; the purchase happens transparently inside the environment's `produce` handling. So Producer inventories of `food/tool` are no longer permanently zero, and GDP's production component reflects real manufacturing activity.

---

5. The Three Headline Metrics, in Detail

5.1 Gini coefficient — what inequality looks like as one number

What it represents: how unevenly wealth is distributed across the population, right now. `0` means perfect equality (everyone has exactly the same wealth). `1` means perfect inequality (one agent has everything, everyone else has nothing). Real-world national Gini coefficients typically range roughly 0.25 (very equal) to 0.63 (very unequal); this simulation's values should be read on that same intuitive scale.

What "wealth" means here (`AgentBase.wealth()`, `aegora_isle/agents/base.py`):

```
wealth = raw_inventory + food_inventory * food_price
                       + tool_inventory * tool_price + cash
```

Every agent's entire holdings, converted to one comparable number using current market prices.

The formula (`kpi_logger.compute_gini_raw`), the standard Lorenz-curve construction:

```
sort everyone's wealth from smallest to largest: w_1 <= w_2 <= ... <= w_n

gini = (n + 1 - 2 * sum(cumulative_sum(w)) / sum(w)) / n
```

Worked examples (these are exact unit tests in `tests/test_kpi.py`):

- Everyone has wealth `100`: `gini = 0.0` (perfect equality).
- Wealths `[0, 0, 0, 100]` (one agent has everything): `gini = 0.75`.

Two versions in the data: `gini_raw` is the instantaneous value, recomputed fresh every cycle (noisy, jumps around). `gini_coefficient` (the "headline" CSV column) is a 50-cycle rolling average of `gini_raw` — smoothed, easier to read a trend from. The live value fed into Governor observations and G3's reward (§2.3.4) uses the **raw**, unsmoothed form, since the Governor needs to react to *right now*, not a trailing average.

5.2 Depletion rate — how fast the lagoon is shrinking, per cycle

What it represents: the percentage of the resource pool that disappeared (or, if negative, regenerated) in the most recent step, relative to how much was there a moment before. It answers "at the current rate, how quickly is this resource running out?"

The formula (`kpi_logger.compute_depletion_rate_raw`):

```
depletion_rate = (stock_at_previous_step - stock_now) / stock_at_previous_step * 100
```

Sign convention (locked deliberately, documented in `DECISIONS.md`): **positive** means the pool shrank (depleting); **negative** means it grew (regenerating faster than it was drawn down, or the resource pool simply ticked up due to logistic regrowth).

Worked example: stock goes from `100` to `95` in one step: `depletion_rate = (100 - 95) / 100 * 100 = 5.0` — a 5% depletion that step. If stock instead rose from 100 to 105: `depletion_rate = (100 - 105) / 100 * 100 = -5.0` — 5% regeneration.

Two versions, like Gini: the CSV's `depletion_rate` column is a 50-cycle rolling average of the per-step percentage above. The *live* value fed into Governor observations and G3's `sustainability` reward term (§2.3.4) is converted to a **fraction** (divided by 100, so 5% becomes `0.05`) specifically so it fits `Governor.observation_space`'s declared `[-1, 1]` bound — both versions describe the same underlying quantity, just scaled differently for their respective consumers.

5.3 Resource stock — the lagoon's current raw-resource quantity

What it represents: the literal, current amount of raw resource sitting in the shared lagoon, in the same units the scenario's `initial_stock` is specified in (e.g. Scenario A starts at `10,000`, Scenario D at `3,000`). This is the single number the entire simulated economy's health hinges on — every Extractor's potential harvest, every disruption event's target, and the collapse trigger are all defined in terms of this one quantity.

What changes it, every cycle (`ResourcePool`, see §4 step 6):

- **Decreases** when any Extractor successfully extracts raw resource (governed by the cap mechanisms in §2.2/§2.3).
- **Decreases sharply** when a random disruption event fires (a 10–30% cut to whatever the current stock is — so the same percentage cut is a much bigger absolute loss when the pool is already large).
- **Increases slightly** every cycle from logistic regeneration, `regeneration_rate * stock * (1 - stock/initial_stock)` — growth is fastest when the pool is around half-full, and tapers to zero as it approaches `initial_stock` (it can never regenerate past its own starting capacity).

Why it matters beyond its own number: `resource_stock` is the input to almost every other KPI in this document — `depletion_rate` is computed directly from it, `stock_norm` (stock ÷ initial_stock) feeds both G3's `sustainability` reward term and every Governor's observation, and `critical_min` (the collapse trigger, scenario-specific — `100` for A/B, `50` for C/D) is compared directly against it every single cycle to decide whether the episode has collapsed.

---

6. Quick reference — file map

Concept	File
Governance dispatcher (all 3 regimes)	`aegora_isle/governance/governance_module.py`
G1 (no-op)	`governance_module.py`'s `G1Handler` class
G2 (static rules)	`aegora_isle/governance/rule_based.py`
G3 (SAC)	`aegora_isle/governance/adaptive_rl.py`
G1/G2/G3 YAML configs	`aegora_isle/governance/configs/*.yaml`
Scenario A/B/C/D parameters	`aegora_isle/experiments/scenarios.yaml`
Lagoon mechanics (stock, regen, events)	`aegora_isle/environment/resource_pool.py`
The AEC cycle itself	`aegora_isle/environment/aegora_env.py`
KPI formulas (Gini, depletion, HDI, GDP)	`aegora_isle/kpi/kpi_logger.py`
PPO training for Extractor/Producer/Trader	`aegora_isle/experiments/train_agents.py`
Action-space flatten/decode for SB3	`aegora_isle/agents/action_codec.py`
Design decisions & known gaps	`DECISIONS.md`